

04_model-event-py

#PostgreSQL #python

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ models/  
            └─ event.py
```

El archivo `event.py` se encuentra dentro de la carpeta de modelos del backend:

```
backend/app/models/
```

Este archivo define el modelo SQLAlchemy `Event`, que representa la tabla de eventos del laboratorio SIEM MVP.

Un evento es una unidad de información recibida por el sistema. En el contexto del laboratorio, puede representar una acción, registro o actividad relevante desde el punto de vista de seguridad.

Este modelo es una de las piezas centrales del flujo de datos:

```
API de ingesta  
  ↓  
Validación del evento  
  ↓  
Modelo Event  
  ↓  
Tabla events en PostgreSQL
```

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,260p' backend/app/models/event.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,260p'
```

Indica que se impriman las líneas de la 1 a la 260.

```
backend/app/models/event.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from typing import Any, Optional
from sqlalchemy.dialects.postgresql import JSONB

from sqlalchemy import DateTime, Integer, String, Text, func
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base


class Event(Base):
    __tablename__ = "events"

    id: Mapped[int] = mapped_column(primary_key=True)
    ts: Mapped[object] = mapped_column(DateTime(timezone=True), server_default=func.now(),
    nullable=False)

    source: Mapped[str] = mapped_column(String(64), nullable=False)
    severity: Mapped[int] = mapped_column(Integer, nullable=False)
    message: Mapped[str] = mapped_column(Text, nullable=False)

    created_at: Mapped[object] = mapped_column(DateTime(timezone=True), server_default=func.now(),
    nullable=False)

    meta: Mapped[Optional[dict[str, Any]]] = mapped_column(JSONB, nullable=True)
```

4 Función general del archivo

El archivo `event.py` define la estructura de los eventos dentro de la base de datos.

La clase principal es:

```
class Event(Base):
```

Esta clase hereda de `Base`, la clase declarativa definida en:

```
backend/app/db/base.py
```

Gracias a esta herencia, SQLAlchemy interpreta `Event` como un modelo ORM, es decir, como una clase Python que representa una tabla de PostgreSQL.

La tabla asociada se llama:

```
__tablename__ = "events"
```

Por tanto, la relación es:

```
Clase Python Event
    ↓
Tabla PostgreSQL events
```

El modelo define los siguientes campos:

<code>id</code>	→ identificador único del evento
<code>ts</code>	→ timestamp del evento
<code>source</code>	→ origen del evento
<code>severity</code>	→ severidad numérica
<code>message</code>	→ mensaje descriptivo
<code>created_at</code>	→ fecha de creación del registro
<code>meta</code>	→ información adicional en formato JSONB

5 Estructura general del archivo

El archivo puede dividirse en cuatro bloques:

```
from typing import Any, Optional
from sqlalchemy.dialects.postgresql import JSONB
```

Importaciones relacionadas con tipos y JSONB.

```
from sqlalchemy import DateTime, Integer, String, Text, func
from sqlalchemy.orm import Mapped, mapped_column
```

Importaciones principales de SQLAlchemy.

```
from app.db.base import Base
```

Importación de la clase base declarativa.

```
class Event(Base):
    ...
```

Definición del modelo `Event`.

Visualmente:

```
event.py
├─ Importaciones de typing
├─ Importación de JSONB
├─ Importaciones SQLAlchemy
├─ Importación de Base
└─ Modelo Event
    ├─ __tablename__
    ├─ id
    ├─ ts
    ├─ source
    ├─ severity
    ├─ message
    ├─ created_at
    └─ meta
```

6 Análisis línea por línea

Importación de `Any` y `Optional`

```
from typing import Any, Optional
```

Esta línea importa dos tipos desde el módulo estándar `typing`.

`Any`

Representa cualquier tipo de dato.

En este archivo se usa para indicar que el diccionario `meta` puede contener valores de cualquier tipo:

```
dict[str, Any]
```

Esto significa:

diccionario cuyas claves son strings y cuyos valores pueden ser de cualquier tipo

```
Optional
```

Indica que un valor puede ser de un tipo concreto o `None`.

En este archivo se usa aquí:

```
Optional[dict[str, Any]]
```

Esto significa:

```
puede ser un diccionario o puede ser None
```

Tiene sentido porque el campo `meta` es opcional.

Importación de `JSONB`

```
from sqlalchemy.dialects.postgresql import JSONB
```

Esta línea importa el tipo `JSONB` específico de PostgreSQL.

`JSONB` permite almacenar datos JSON de forma binaria y eficiente en PostgreSQL.

En este archivo se usa para el campo:

```
meta
```

La ventaja de `JSONB` es que permite guardar información flexible sin tener que crear una columna específica para cada dato adicional.

Ejemplo de `meta`:

```
{
    "ip": "192.168.1.10",
    "user": "admin",
    "action": "login_failed"
}
```

En un laboratorio SIEM esto es útil porque no todos los eventos tienen exactamente los mismos campos.

Importaciones principales de `SQLAlchemy`

```
from sqlalchemy import DateTime, Integer, String, Text, func
```

Esta línea importa varios tipos y funciones de `SQLAlchemy`.

`DateTime`

```
DateTime
```

Representa una columna de fecha y hora.

En este archivo se usa en:

```
ts
created_at
```

`Integer`

```
Integer
```

Representa una columna numérica entera.

En este archivo se usa en:

```
severity
```

String

```
String
```

Representa una cadena de texto con longitud limitada.

En este archivo se usa en:

```
source
```

con una longitud máxima de 64 caracteres.

Text

```
Text
```

Representa una cadena de texto larga.

En este archivo se usa en:

```
message
```

Tiene sentido porque el mensaje del evento puede ser más largo que un campo `String` corto.

func

```
func
```

Permite usar funciones SQL desde SQLAlchemy.

En este archivo se usa:

```
func.now()
```

Esto representa la función SQL `NOW()`, utilizada para generar fechas automáticamente desde la base de datos.

Importación de `Mapped` y `mapped_column`

```
from sqlalchemy.orm import Mapped, mapped_column
```

Esta línea importa dos elementos del ORM moderno de SQLAlchemy.

Mapped

```
Mapped
```

Se utiliza para anotar los atributos del modelo.

Ejemplo:

```
id: Mapped[int]
```

Esto indica que `id` es una columna mapeada por SQLAlchemy y que su tipo Python esperado es `int`.

mapped_column

```
mapped_column
```

Se utiliza para definir una columna de base de datos dentro de un modelo ORM.

Ejemplo:

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Esto define una columna `id` como clave primaria.

Importación de Base

```
from app.db.base import Base
```

Esta línea importa la clase `Base` definida en:

```
backend/app/db/base.py
```

`Base` es la clase declarativa común de los modelos SQLAlchemy.

La clase `Event` hereda de ella:

```
class Event(Base):
```

Esto permite que SQLAlchemy registre `Event` como modelo ORM.

Definición de la clase Event

```
class Event(Base):
```

Esta línea define la clase `Event`.

Desglose:

```
class
```

Palabra clave de Python para definir una clase.

```
Event
```

Nombre de la clase.

Representa un evento de seguridad dentro del laboratorio.

```
(Base)
```

Indica que `Event` hereda de `Base`.

Esto convierte la clase en un modelo ORM declarativo de SQLAlchemy.

Nombre de la tabla

```
__tablename__ = "events"
```

Esta línea indica el nombre de la tabla en la base de datos.

Desglose:

```
__tablename__
```

Atributo especial usado por SQLAlchemy para saber qué tabla representa esta clase.

```
"events"
```

Nombre real de la tabla PostgreSQL.

Por tanto:

```
Event → events
```

Campo `id`

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Esta línea define la columna `id`.

Desglose:

```
id
```

Nombre del atributo en Python y de la columna en la tabla.

```
: Mapped[int]
```

Indica que es una columna mapeada y que su tipo esperado es `int`.

```
= mapped_column(...)
```

Define la columna usando SQLAlchemy.

```
primary_key=True
```

Indica que esta columna es la clave primaria de la tabla.

La clave primaria identifica de forma única cada registro.

Conceptualmente:

```
id → identificador único del evento
```

Campo `ts`

```
ts: Mapped[object] = mapped_column(DateTime(timezone=True), server_default=func.now(),  
nullable=False)
```

Esta línea define la columna `ts`.

`ts` significa normalmente `timestamp`.

Representa la fecha y hora asociada al evento.

Desglose:

```
ts
```

Nombre del atributo y de la columna.

```
Mapped[object]
```

Indica que es una columna mapeada. Se usa `object` como tipo genérico para el valor de fecha/hora.

```
mapped_column(...)
```

Define la columna.

Tipo `DateTime(timezone=True)`

```
DateTime(timezone=True)
```

Indica que la columna almacena fecha y hora con información de zona horaria.

Esto es importante en sistemas de eventos, porque los logs pueden proceder de distintos sistemas o momentos.

Valor por defecto `server_default=func.now()`

```
server_default=func.now()
```

Indica que si no se proporciona un valor para `ts`, la base de datos asignará automáticamente la hora actual.

La parte `server_default` significa que el valor por defecto lo aplica el servidor de base de datos, no Python.

```
func.now()
```

Representa la función SQL `NOW()`.

Restricción `nullable=False`

```
nullable=False
```

Indica que la columna no puede ser nula.

Todo evento debe tener un timestamp.

Campo `source`

```
source: Mapped[str] = mapped_column(String(64), nullable=False)
```

Esta línea define la columna `source`.

Representa el origen del evento.

Ejemplos posibles:

```
firewall
auth
linux
windows
web
ids
```

Desglose:


```
source
```

Nombre del atributo y de la columna.

```
Mapped[str]
```

Indica que el tipo Python esperado es `str`.

```
String(64)
```

Define una cadena de texto con máximo 64 caracteres.

```
nullable=False
```

Indica que el campo es obligatorio.

Todo evento debe tener un origen.

Campo `severity`

```
severity: Mapped[int] = mapped_column(Integer, nullable=False)
```

Esta línea define la columna `severity`.

Representa la severidad del evento.

Desglose:

```
severity
```

Nombre del atributo y de la columna.

```
Mapped[int]
```

Tipo Python esperado: entero.

```
Integer
```

Tipo de columna SQL: número entero.

```
nullable=False
```

Campo obligatorio.

La severidad permite clasificar la importancia del evento.

Ejemplo conceptual:

```
1 → baja  
2 → media  
3 → alta  
4 → crítica
```

La escala exacta depende de cómo se haya diseñado el sistema.

Campo `message`

```
message: Mapped[str] = mapped_column(Text, nullable=False)
```

Esta línea define la columna `message`.

Representa el mensaje descriptivo del evento.

Desglose:

```
message
```

Nombre del atributo y de la columna.

```
Mapped[str]
```

Tipo Python esperado: string.

```
Text
```

Tipo SQL para texto largo.

```
nullable=False
```

Campo obligatorio.

Este campo puede contener una descripción del evento, por ejemplo:

```
Failed login attempt for user admin
Connection blocked from suspicious IP
Multiple authentication failures detected
```

Campo `created_at`

```
created_at: Mapped[object] = mapped_column(DateTime(timezone=True), server_default=func.now(),
nullable=False)
```

Esta línea define la columna `created_at`.

Representa la fecha y hora en la que el evento fue creado dentro de la base de datos.

Aunque se parece a `ts`, no significa necesariamente lo mismo.

Diferencia conceptual:

```
ts          → momento asociado al evento
created_at → momento en que el registro se crea en la base de datos
```

En muchos casos pueden coincidir, pero no siempre.

Por ejemplo, un evento podría haber ocurrido a las 10:00, pero ser recibido e insertado en el SIEM a las 10:05.

Tipo `DateTime(timezone=True)`

```
DateTime(timezone=True)
```

Fecha y hora con zona horaria.

Valor por defecto

```
server_default=func.now()
```

Si no se proporciona `created_at`, PostgreSQL asigna la hora actual.

Restricción

```
nullable=False
```

El campo no puede ser nulo.

Campo meta

```
meta: Mapped[Optional[dict[str, Any]]] = mapped_column(JSONB, nullable=True)
```

Esta línea define la columna `meta`.

Es uno de los campos más flexibles del modelo.

Permite almacenar información adicional del evento en formato JSONB.

Desglose:

```
meta
```

Nombre del atributo y columna.

```
Mapped[Optional[dict[str, Any]]]
```

Indica que el valor puede ser:

- Un diccionario con claves `string` y valores de cualquier tipo.
- `None`.

```
mapped_column(JSONB, nullable=True)
```

Define una columna PostgreSQL de tipo JSONB y permite valores nulos.

Por qué usar JSONB

En un laboratorio SIEM, no todos los eventos tienen los mismos campos.

Un evento de autenticación puede tener:

```
{
  "user": "admin",
  "ip": "192.168.1.10"
}
```

Un evento de firewall puede tener:

```
{
  "src_ip": "10.0.0.5",
  "dst_port": 443,
  "action": "blocked"
}
```

Usar `meta` permite guardar esos datos adicionales sin modificar la estructura principal de la tabla.

Resultado final del archivo

Después de cargar este archivo, SQLAlchemy dispone del modelo:

```
Event
```

Este modelo representa la tabla:

```
events
```

con los campos:

```
id
ts
source
severity
message
created_at
meta
```

7 Relación con el flujo técnico del laboratorio

El modelo `Event` participa directamente en el flujo principal del SIEM.

La relación técnica sería:

```
POST /ingest
  ↓
datos recibidos por FastAPI
  ↓
validación mediante schema
  ↓
creación de objeto Event
  ↓
SQLAlchemy guarda Event
  ↓
registro insertado en tabla events
```

Después, esos eventos pueden ser consultados por otros endpoints:

```
GET /events
GET /metrics
```

También pueden ser evaluados por el motor de reglas para generar alertas.

Relación general:

```
Event
  ↓
representa evento recibido
  ↓
se almacena en PostgreSQL
  ↓
puede activar Rule
  ↓
puede generar Alert
```

8 Errores típicos o puntos importantes

Diferencia entre `ts` y `created_at`

Aunque ambos son campos de fecha, no tienen por qué significar lo mismo.

```
ts          → momento del evento
created_at → momento de inserción en la base de datos
```

En este proyecto ambos tienen valor por defecto `func.now()`, pero conceptualmente conviene diferenciarlos.

meta puede ser nulo

El campo `meta` está definido como:

```
nullable=True
```

Esto significa que un evento puede no tener metadatos adicionales.

source, **severity** y **message** son obligatorios

Estos campos tienen:

```
nullable=False
```

Por tanto, no pueden insertarse eventos sin esos valores.

JSONB es específico de PostgreSQL

El tipo:

```
JSONB
```

pertenece al dialecto PostgreSQL.

Esto significa que el modelo está diseñado específicamente para PostgreSQL, no para cualquier base de datos genérica.

Longitud máxima de **source**

El campo `source` usa:

```
String(64)
```

Por tanto, el origen del evento no debería superar los 64 caracteres.

Mapped y **mapped_column** pertenecen al estilo moderno de SQLAlchemy

Este modelo usa sintaxis moderna de SQLAlchemy 2.x:

```
id: Mapped[int] = mapped_column(...)
```

Esta forma combina tipado Python con definición ORM.

9 Comandos útiles relacionados

Comprobar que el modelo se puede importar:

```
docker exec -it siem-api python -c "from app.models.event import Event; print(Event)"
```

Comprobar el nombre de la tabla:

```
docker exec -it siem-api python -c "from app.models.event import Event; print(Event.__tablename__)"
```

Comprobar columnas del modelo:

```
docker exec -it siem-api python -c "from app.models.event import Event;
print(Event.__table__.columns.keys())"
```

Comprobar tablas en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "\dt"
```

Consultar estructura de la tabla events :

```
docker exec -it siem-db psql -U siem -d siem -c "\d events"
```

Consultar eventos almacenados:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, ts, source, severity, message, created_at, meta FROM events LIMIT 10;"
```

Contar eventos:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT COUNT(*) FROM events;"
```

Consultar eventos por severidad:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT severity, COUNT(*) FROM events GROUP BY severity ORDER BY severity;"
```

Consultar eventos con metadatos:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT id, meta FROM events WHERE meta IS NOT NULL LIMIT 10;"
```

Dataview (inline field 'mapped_column(...)'): Error:

-- PARSING FAILED -----

```
> 1 | mapped_column(...)
    |                   ^
```

Expected one of the following:

'(', ')', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable